

計算機科学実験及び演習 3 HW

機能設計仕様書

7 班

構成員: 甲斐 豪、黒山 理恵、芝原 遼太

文責: 芝原遼太

学籍番号 1029-36-9619 入学年度 2024 年

提出期限:2025 年 6 月 5 日

提出日:2025 年 6 月 5 日

コンポーネントとしては、ALU、シフタ、主記憶、16 ビットレジスタ、4 ビットレジスタ、汎用レジスタ、プログラムカウンタ、外部出力装置、符号拡張、(マルチプレクサ)、パイプラインレジスタ、制御ユニット、フォワーディングユニット、ハザード検出ユニットに分割した。

今回、私はレジスタファイル（汎用レジスタ）、外部出力装置、ハザード検出ユニットの設計と実装を担当した。それぞれの機能設計を以下に説明する。

1.2.レジスタファイル

16bit 幅の汎用レジスタを 8 本持ち、外部から指定された 2 つのレジスタの値を同時に読み出し、指定された 1 つのレジスタにデータを書き込む機能を持つ。

信号線名とそれぞれの説明は以下の表 2 のとおりである。

信号線名	ビット数	入出力	機能
clk	1bit	入力	書き込み動作の基準となるクロック信号
rst	1bit	入力	負論理リセット信号。0 になると全レジスタを 0 に初期化する
we	1bit	入力	書き込み許可信号。1 で書き込みを行い、0 で書き込まない
rs	3bit	入力	第 1 読み出しレジスタ番号
rt	3bit	入力	第 2 読み出しレジスタ番号
rd	3bit	入力	書き込み先レジスタ番号
wd	16bit	入力	書き込みデータ
rs_data	16bit	出力	rs で指定されたレジスタの値
rt_data	16bit	出力	rt で指定されたレジスタの値

表 2

ソースコードを以下に示す。

```
module regfile (  
    input phase,  
    input clk,  
    input rst,  
    input we,          // write enable  
    input [2:0] rs,  
    input [2:0] rt,  
    input [2:0] rd,    // 書き込み先  
    input [15:0] wd,   // 書き込みデータ  
    output [15:0] rs_data,  
    output [15:0] rt_data  
);  
  
    reg [15:0] regs [0:7]; // 8 本のレジスタ  
  
    integer i;  
  
    assign rs_data = regs[rs];  
    assign rt_data = regs[rt];  
  
    always @(posedge clk) begin  
        if (!rst) begin  
            for (i = 0; i < 8; i = i + 1) begin  
                regs[i] <= 16'b0;  
            end  
        end  
    end
```

```

        end
    end else begin
        if (phase&&we) begin
            regs[rd] <= wd;
        end
    end
end
end
endmodule

```

1.3.外部出力

OUT 命令によって出力対象となる 16bit データを受け取り、これを 16 進数 4 桁としてダイナミック点灯を用いて 4 桁 7 セグメント LED に表示する回路である。外部からは、表示対象データ、クロック、リセットを受け取り、7 セグメント LED のセグメント信号とセレクト信号を出力する。

4 つのモジュールからなり、LED モジュールは、4bit の 2 進数を対応する 7 セグメント LED 表示パターンへ変換する組み合わせ回路である。output7seg4 モジュールは、16bit 入力データを 4bit ずつ 4 つに分割し、それぞれを LED モジュールへ入力することで、16 進数 4 桁分の 7 セグメント表示パターンを生成する役割を持つ。dynamic モジュールは、4 桁分の表示パターンを高速に切り替えながら順番に表示するダイナミック点灯制御回路である。top モジュールはシステム全体を接続する最上位モジュールである。

以下にソースコードを示す。

```

module top (
    input clk,
    input rst,
    input [15:0] data,

    output [7:0] seg,
    output [3:0] sel
);

wire [7:0] hex3;
wire [7:0] hex2;
wire [7:0] hex1;
wire [7:0] hex0;

output7seg4 u_output7seg4 (
    .data(data),
    .hex3(hex3),
    .hex2(hex2),
    .hex1(hex1),
    .hex0(hex0)
);

dynamic u_dynamic (
    .clk(clk),
    .rst(rst),

```

<pre> .hex3(hex3), .hex2(hex2), .hex1(hex1), .hex0(hex0), .seg(seg), .sel(sel)); endmodule </pre>	
---	--

<pre> module LED (input [3:0] in, output [7:0] out //output selec); assign out = (in == 4'b0000) ? 8'b1111_1100 : (in == 4'b0001) ? 8'b0110_0000 : (in == 4'b0010) ? 8'b1101_1010 : (in == 4'b0011) ? 8'b1111_0010 : (in == 4'b0100) ? 8'b0110_0110 : (in == 4'b0101) ? 8'b1011_0110 : (in == 4'b0110) ? 8'b1011_1110 : (in == 4'b0111) ? 8'b1110_0000 : (in == 4'b1000) ? 8'b1111_1110 : (in == 4'b1001) ? 8'b1111_0110 : (in == 4'b1010) ? 8'b1110_1110 : (in == 4'b1011) ? 8'b0011_1110 : (in == 4'b1100) ? 8'b0001_1010 : (in == 4'b1101) ? 8'b0111_1010 : (in == 4'b1110) ? 8'b1001_1110 : (in == 4'b1111) ? 8'b1000_1110 : 8'b0000_0000 ; //assign selec = 1'b0; endmodule </pre>	
--	--

<pre> module output7seg4 (input [15:0] data, output [7:0] hex3, output [7:0] hex2, output [7:0] hex1, output [7:0] hex0); LED led3 (.in(data[15:12]), .out(hex3)); LED led2 (.in(data[11:8]), .out(hex2)); </pre>	
---	--

```

LED led1 (
    .in(data[7:4]),
    .out(hex1)
);

LED led0 (
    .in(data[3:0]),
    .out(hex0)
);

endmodule

```

```

module dynamic (
    input clk,
    input rst,

    input [7:0] hex3,
    input [7:0] hex2,
    input [7:0] hex1,
    input [7:0] hex0,

    output reg [7:0] seg,
    output reg [3:0] sel
);

    reg [15:0] count;
    reg [1:0] digit;

    always @(posedge clk or negedge rst) begin
        if (!rst) begin
            count <= 16'd0;
            digit <= 2'd0;
        end else begin
            if (count == 16'd4999) begin
                count <= 16'd0;
                digit <= digit + 2'd1;
            end else begin
                count <= count + 16'd1;
            end
        end
    end

    always @(*) begin
        case (digit)
            2'd0: begin
                seg = hex0;
                sel = 4'b1110;
            end
            2'd1: begin
                seg = hex1;
                sel = 4'b1101;
            end
        end
    end
end

```

```

2'd2: begin
    seg = hex2;
    sel = 4'b1011;
end
2'd3: begin
    seg = hex3;
    sel = 4'b0111;
end
default: begin
    seg = 8'b0000_0000;
    sel = 4'b1111;
end
endcase
end
endmodule

```

1.3.ハザード検出ユニット

本ハザード検出ユニットは、パイプライン実行時に発生するデータハザードを検出し、必要に応じてストールを発生させる回路である。

入力として、前の命令が LW または IN 命令であることを示す memtoreg、現在の命令コード I、および前の命令の書き込み先レジスタ番号 Rt を受け取る。命令コードから命令種別と参照レジスタを抽出し、現在の命令が使用するレジスタと前の命令の書き込み先レジスタを比較する。

memtoreg がアサートされており、前の命令の書き込み先レジスタと現在の命令の参照レジスタが一致した場合、ハザードが発生していると判断する。このとき、PCwrite と IFIDwrite を 0 にして PC および IF/ID パイプラインレジスタの更新を停止し、stall を 1 にすることでバブルを挿入する。

これにより、ロード命令や外部入力命令の結果が利用可能になるまで後続命令を 1 サイクル待機させ、誤ったデータ参照を防止する。

以下にソースコードを示す。

```

module HazardDetect(
    input memtoreg, //IN か LW でアサート、データハザードを起こす命令の検知
    input [15:0] I, //命令 16bit コード
    input [2:0] Rt, //前の命令の書き込み先
    output reg PCwrite, //PC 書き込み許可信号
    output reg IFIDwrite, //IF/ID 書き込み許可信号
    output reg stall //ストール信号
);
    wire [1:0] op=I[15:14]; //命令の分類
    wire [2:0] reg1=I[13:11]; //レジスタ指定フィールド
    wire [2:0] reg2=I[10:8]; //レジスタ指定フィールド
    wire [3:0] op3=I[7:4]; //演算種別

```

```

always @(*) begin // デフォルトではストールしない
PCwrite=1'b1;
IFIDwrite=1'b1;
stall=1'b0;
if(({op,op3}==6'b110000)|| //ADD
    ({op,op3}==6'b110001)|| //SUB
    ({op,op3}==6'b110010)|| //AND
    ({op,op3}==6'b110011)|| //OR
    ({op,op3}==6'b110100)|| //EXOR
    ({op,op3}==6'b110101)|| //CMP
    (op==2'b01) //SW
    ) begin //reg1,reg2 両方使う 命令
if(memtoreg) begin
    if(Rt==reg1||Rt==reg2) begin // PC と IF/ID レジスタを保持し、ID/EX 段へバブルを挿入する

        PCwrite=1'b0;
        IFIDwrite=1'b0;
        stall=1'b1;
    end
end
end
else if(({op,op3}==6'b110110)|| //MOV
    ({op,op3}==6'b111101) //OUT
    ) begin //reg1 のみ使う 命令
if(memtoreg) begin
    if(Rt==reg1) begin
        PCwrite=1'b0;
        IFIDwrite=1'b0;
        stall=1'b1;
    end
end
end
end
end

```



```
else if(({op,op3}==6'b110111)|| //ADDi(Rd+即値)
    ({op,op3}==6'b111000)|| //shift↓
    ({op,op3}==6'b111001)||
    ({op,op3}==6'b111010)||
    ({op,op3}==6'b111011)||
    (op==2'b00) //LW
    ) begin //reg2 のみ使う 命令
if(memtoreg) begin
    if(Rt==reg2) begin //Load-Use ハザード検出
        PCwrite=1'b0;
        IFIDwrite=1'b0;
        stall=1'b1;
    end
end
end
end
endmodule
```

2.性能評価

2.1.レジスタファイル

上記のレジスタファイル単体について、Quartus Prime 20.1 を用いて論理合成および性能評価を行った。

図 3 にコンパイル結果の概要を、図 4 に FPGA 資源使用量の詳細を示す。

Flow Status	Successful - Thu May 28 13:43:01 2026
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Standard Edition
Revision Name	regfile
Top-level Entity Name	regfile
Family	Cyclone IV E
Total logic elements	168 / 6,272 (3 %)
Total registers	128
Total pins	60 / 92 (65 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP4CE6E22C6
Timing Models	Final

図 3

	Resource	Usage
1	▼ Total logic elements	168 / 6,272 (3 %)
1	-- Combinational with no register	40
2	-- Register only	0
3	-- Combinational with a register	128
2		
3	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	136
2	-- 3 input functions	32
3	-- <=2 input functions	0
4	-- Register only	0
4		
5	▼ Logic elements by mode	
1	-- normal mode	168
2	-- arithmetic mode	0
6		
7	▼ Total registers*	128 / 6,684 (2 %)
1	-- Dedicated logic registers	128 / 6,272 (2 %)
2	-- I/O registers	0 / 412 (0 %)
8		
9	Total LABs: partially or completely used	12 / 392 (3 %)
10	Virtual pins	0
11	▼ I/O pins	60 / 92 (65 %)
1	-- Clock pins	2 / 3 (67 %)
2	-- Dedicated input pins	0 / 9 (0 %)
12		
13	M9Ks	0 / 30 (0 %)
14	Total block memory bits	0 / 276,480 (0 %)
15	Total block memory implementation bits	0 / 276,480 (0 %)
16	Embedded Multiplier 9-bit elements	0 / 30 (0 %)
17	PLLs	0 / 2 (0 %)
18	▼ Global signals	2
1	-- Global clocks	2 / 10 (20 %)
19	JTAGs	0 / 1 (0 %)
20	CRC blocks	0 / 1 (0 %)
21	ASMI blocks	0 / 1 (0 %)
22	Oscillator blocks	0 / 1 (0 %)
23	Impedance control blocks	0 / 4 (0 %)
24	Average interconnect usage (total/H/V)	1.1% / 1.0% / 1.2%
25	Peak interconnect usage (total/H/V)	2.5% / 2.4% / 2.7%
26	Maximum fan-out	128
27	Highest non-global fan-out	48
28	Total fan-out	1251
29	Average fan-out	2.92

図 4

合成結果より、本レジスタファイルは 168 個の Logic Elements を使用し、全体使用率は 3% であった。また、128 個のレジスタを使用した。これは、16bit 幅のレジスタを 8 本保持する構成であるためである。

また、論理要素の内訳としては、4 入力 LUT を用いた組み合わせ回路が 136 個使用されている。これは主に、読み出しレジスタ選択回路および書き込み制御回路に使用されていると考えられる。

本回路では、読み出し処理は組み合わせ回路として実装されている。そのため、レジスタ番号入力 rs, rt に応じて、8 本の 16bit レジスタから対象データを選択する多重化回路が構成される。この読み出し選択回路が遅延の主な要因となる。

一方、書き込み処理はクロック同期で行われるため、書き込みタイミングは安定しており、高速動作に適した構成となっている。

以上より、本レジスタファイルは比較的小規模な回路規模で実装できており、SIMPLE/B プロセッサにおける汎用レジスタ機能を十分に実現できることを確認した。

2.2 外部出力

外部出力装置についても、性能評価を行った。

図 5 にコンパイル結果の概要を、図 6 に FPGA 資源使用量の詳細を示す。

Flow Status	Successful - Fri May 29 09:42:54 2026
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Standard Edition
Revision Name	output
Top-level Entity Name	top
Family	Cyclone IV E
Total logic elements	74 / 6,272 (1 %)
Total registers	18
Total pins	30 / 92 (33 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP4CE6E22C6
Timing Models	Final

図 5

	Resource	Usage
1	▼ Total logic elements	74 / 6,272 (1 %)
1	-- Combinational with no register	56
2	-- Register only	0
3	-- Combinational with a register	18
2		
3	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	47
2	-- 3 input functions	1
3	-- <=2 input functions	26
4	-- Register only	0
4		
5	▼ Logic elements by mode	
1	-- normal mode	59
2	-- arithmetic mode	15
6		
7	▼ Total registers*	18 / 6,684 (< 1 %)
1	-- Dedicated logic registers	18 / 6,272 (< 1 %)
2	-- I/O registers	0 / 412 (0 %)
8		
9	Total LABs: partially or completely used	5 / 392 (1 %)
10	Virtual pins	0
11	▼ I/O pins	30 / 92 (33 %)
1	-- Clock pins	2 / 3 (67 %)
2	-- Dedicated input pins	0 / 9 (0 %)
12		
13	M9Ks	0 / 30 (0 %)
14	Total block memory bits	0 / 276,480 (0 %)
15	Total block memory implementation bits	0 / 276,480 (0 %)
16	Embedded Multiplier 9-bit elements	0 / 30 (0 %)
17	PLLs	0 / 2 (0 %)
18	▼ Global signals	2
1	-- Global clocks	2 / 10 (20 %)
19	JTAGs	0 / 1 (0 %)
20	CRC blocks	0 / 1 (0 %)
21	ASMI blocks	0 / 1 (0 %)
22	Oscillator blocks	0 / 1 (0 %)
23	Impedance control blocks	0 / 4 (0 %)
24	Average interconnect usage (total/H/V)	0.2% / 0.2% / 0.2%
25	Peak interconnect usage (total/H/V)	1.1% / 0.9% / 1.2%
26	Maximum fan-out	18
27	Highest non-global fan-out	16
28	Total fan-out	344
29	Average fan-out	2.10

図 6

合成結果より、本回路は 74 個の Logic Elements を使用し、FPGA 全体に対する使用率は 1% であった。また、18 個のレジスタを使用した。

レジスタ数が少ない理由は、本回路において状態を保持する要素が dynamic モジュール内にしかなく、大部分が組み合わせ回路によって構成されているためである。

論理要素の内訳を見ると、4 入力 LUT が 47 個、3 入力 LUT が 1 個、2 入力以下の LUT が 26 個使用されている。これらは主に LED モジュール内の 4bit から 7 セグメント LED パターンへの変換回路、および dynamic モジュール内の桁選択制御回路として利用されている。

LED モジュールおよび output7seg4 モジュールは組み合わせ回路であり、遅延要因は主に 4bit 入力から 7 セグメント出力を生成するデコーダ回路である。一方、dynamic モジュールでは 16bit カウンタを用いて表示桁の切り替えを行っているため、カウンタ加算回路が順序回路部分の主要な遅延要因となる。以上の結果から、本外部出力装置は 74 Logic Elements、18 レジスタという小規模な回路規模で実装できており、SIMPLE/B プロセッサに対して十分低コストで外部表示機能を提供できることが確認できた。

また、使用資源率は FPGA 全体の 1% 程度であり、プロセッサ本体の実装に対してほとんど影響を与えないことが分かった。

2.3 ハザード検出ユニット

ハザード検出ユニットについて、性能評価を行った。

図 7 にコンパイル結果の概要を、図 8 に FPGA 資源使用量の詳細を示す。

Flow Status	Successful - Thu Jun 4 18:01:37 2026
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Standard Edition
Revision Name	Hazard
Top-level Entity Name	Hazard
Family	Cyclone IV E
Device	EP4CE30F23I7
Timing Models	Final
Total logic elements	15 / 28,848 (< 1 %)
Total registers	0
Total pins	23 / 329 (7 %)
Total virtual pins	0
Total memory bits	0 / 608,256 (0 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	0 / 4 (0 %)

図 7

	Resource	Usage
1	Estimated Total logic elements	15
2		
3	Total combinational functions	15
4	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	10
2	-- 3 input functions	5
3	-- <=2 input functions	0
5		
6	▼ Logic elements by mode	
1	-- normal mode	15
2	-- arithmetic mode	0
7		
8	▼ Total registers	0
1	-- Dedicated logic registers	0
2	-- I/O registers	0
9		
10	I/O pins	23
11		
12	Embedded Multiplier 9-bit elements	0
13		
14	Maximum fan-out node	alw...0~1
15	Maximum fan-out Maximum fan-out node	4
16	Total fan-out	81
17	Average fan-out	1.33

図 8

合成結果より、本回路は 15 個の Logic Elements を使用し、FPGA 全体に対する使用率は 1% 未満であった。また、レジスタは使用しておらず、回路全体が組み合わせ回路として実装されていることが確認できた。

論理要素の内訳を見ると、4 入力 LUT が 10 個、3 入力 LUT が 5 個使用されている。これは主に、命令中のレジスタ番号と先行命令の書き込み先レジスタ番号との比較処理、およびストール信号生成のための論理判定回路に使用されている。

本回路では、IF/ID 段の命令情報と、先行命令の書き込み先レジスタ情報を比較し、データハザードの有無を判定する。比較対象となる信号数が少なく、演算内容もレジスタ番号の一致判定および論理積・論理和演算が中心であるため、非常に小規模な回路となっている。

また、レジスタを持たない組み合わせ回路であることから、ハザード判定結果は入力信号の変化に応じて即座に出力へ反映される構成となっている。

以上より、ハザード検出ユニットは少ない FPGA 資源で実現できており、パイプライン動作に必要なハザード検出機能を低コストで提供できることが確認できた。

3.考察及び感想

本実験では、SIMPLE/B プロセッサをベースとして、5 段パイプライン化、ハーバードアーキテクチャ化、簡易分岐予測機構などを追加したプロセッサの設計・実装を行った。講義ではパイプラインによる高速化の原理を学んでいたが、実際に回路として実装することで、その効果や課題を理解できた。

設計全体について考察すると、パイプライン化によって命令の並列実行が可能になり、スループット向上が期待できる一方で、フォワーディングユニットやハザード検出ユニットを追加しなければ正しい動作が保証できず、高性能化と引き換えに回路構成が大幅に複雑化することが分かった。また、命令メモリとデータメモリを分離するハーバードアーキテクチャの採用も事実上必要で、これにより命令フェッチとデータアクセスの競合を回避でき、構造ハザードの削減に有効であることを確認できた。これらの設計を通じて、現代のプロセッサが高性能化のために多くの補助回路を必要としている理由を理解することができた。

私が担当したレジスタファイルについては、8 本の 16bit レジスタを持ち、2 ポート読み出しと 1 ポート書き込みを実現した。性能評価では 168 個の Logic Elements と 128 個のレジスタを使用した。FPGA 全体から見ると小規模でありながら、プロセッサ全体のデータ受け渡しを担う重要な回路であることを再認識した。特に、パイプライン化されたプロセッサでは各命令が同時進行で動作するため、レジスタへの読み出し・書き込みタイミングが非常に重要であり、基本的な回路であってもシステム全体の動作に大きく影響することを学んだ。

外部出力回路については、CPU 内部の 16bit データを 4 桁の 7 セグメント LED へ表示する機能を実装した。回路規模は 74 個の Logic Elements と 18 個のレジスタと小規模であり、プロセッサ本体にほとんど負荷を与えずに実装できることが分かった。また、ダイナミック点灯方式を利用することで限られた出力ピン数でも 4 桁表示を実現でき、ハードウェア資源を効率的に利用する設計の重要性を学んだ。さらに、CPU 内部で処理されている値を直接確認できるため、デバッグや動作確認の面でも有効な回路であると感じた。

ハザード検出ユニットについては、LW 命令や IN 命令によって発生するデータハザードを検出し、必要に応じてストールを発生させる回路を設計した。性能評価の結果、15 個の Logic Elements のみで実装できており、非常に低コストでハザード対策を実現できることが分かった。一方で、実際に設計してみると命令形式ごとに参照レジスタが異なるため、どのレジスタが使用されるのかを正しく判定する必要があり、想像以上に複雑であった。また、ハザードを見逃すと誤った演算結果につながるため、性能だけでなく正確性も重要であることを実感した。

本実験を通して、CPU は単純な演算回路の集合ではなく、性能向上のための多くの制御回路や補助回路によって成り立っていることを学んだ。特に、パイプライン化による高速化と、それに伴うハザード処理や制御回路の複雑化とのトレードオフを実際に体験できたことは大きな収穫であった。特にハザード検出ユニットに関しては、複雑な部分はあるものの、パイプライン化に極めて重要なモジュールが比較的小規模な組み合わせ回路で実装できたことで優れたアイデアであることが体感できた。また、これまで講義で学んできたプロセッサアーキテクチャの知識を、実際の FPGA 上で動作する回路として実装できたことで理解がより深まった。